University of Pisa

Parallel and Distributed Systems

# Optimization of Partitioned Hash Join with Duplicates

Assignment 1 - Report

Gianluca Panzani

April 4, 2026

# Contents

**1 Implementation** **1**
1.1 Execution Variants . . . . . 1
1.2 Grid Search approach . . . . . 1

**2 SIMD Optimization** **2**
2.1 Compiler evidence of vectorization . . . . . 2
2.2 AVX2 mask hash implementation . . . . . 2

**3 Results Analysis** **3**
3.1 Throughput . . . . . 3
3.2 Partition-time . . . . . 4
3.3 Speedup . . . . . 4

**4 Conclusions** **5**

# 1 Implementation

The implemented project focuses on the partition phase of a Partitioned Hash Join under duplicate-heavy workloads.

## 1.1 Execution Variants

The codebase provides a reproducible benchmarking pipeline to compare three execution variants: `plain_novec`, `plain_vec`, and `avx2`. Each variant is got by a different compilation of the same file (`main.cpp`) and applies the same high-level algorithm but uses different optimization strategies.

The executable targets can be built with the commands: `make plain_novec`, `make plain_vec` and `make avx2`. The corresponding compiler invocations defined in the Makefile are:

```
g++ -std=c++17 -Wall -Wextra -I. -O3 -march=native -fno-tree-vectorize \
    -o plain_novec main.cpp lib/dataset.cpp lib/timing.cpp lib/checksum.cpp \
    lib/results.cpp lib/partition.cpp lib/config.cpp

g++ -std=c++17 -Wall -Wextra -I. -O3 -march=native -mavx2 \
    -fopt-info-vec-optimized -fopt-info-vec-missed \
    -o plain_vec main.cpp lib/dataset.cpp lib/timing.cpp lib/checksum.cpp \
    lib/results.cpp lib/partition.cpp lib/config.cpp

g++ -std=c++17 -Wall -Wextra -I. -O3 -march=native -mavx2 \
    -fopt-info-vec-optimized -fopt-info-vec-missed -DENABLE_AVX2 \
    -o avx2 main.cpp lib/dataset.cpp lib/timing.cpp lib/checksum.cpp \
    lib/results.cpp lib/partition.cpp lib/config.cpp lib/partition_avx2.cpp
```

Executing the programs as defined in the `README.md` file, they receive the same command-line arguments, which include:

- the dataset size `N`,
- the number of partitions `P`,
- the name of the hash function to be used,
- the output CSV file path.

The runtime workflow includes: loading the two input datasets, partitioning both relations with the selected hash function, computing throughput and checksums, writing partitioned binary outputs and appending one result row to the CSV file. This row contains the full experiment metadata (`N`, `P`, hash, execution type), timing metrics (partition time, global time) and other fields (like throughput and checksum), which are then used by the analysis notebooks to extract statistics.

## 1.2 Grid Search approach

The experimental automation is implemented with Bash orchestrators and Slurm runners. The script `run_grid_search.sh` performs:

- cleanup (`make cleanall`) and compilation (`make`),
- dataset generation job submission,
- chained Slurm submissions with dependencies for `plain_novec` → `plain_vec` → `avx2`.

The search space is defined in `runners/grid_config.sh`: $\texttt{N} \in \{10^5, 10^6, 10^7\}$, $\texttt{P} \in \{16, \ldots, 4096\}$ (powers of two), hash $\in \{\texttt{mask}, \texttt{mul}, \texttt{fmix64}\}$. This gives 81 configurations per executable for each full sweep.

The script `stability_benchmark.sh` uses `runners/grid_config_2.sh` (`N` fixed to $10^7$) and repeats the same parameter grid `M` times (in the current dataset, 5 repetitions). This is used to separate raw performance from run-to-run variance.

An important implementation detail is that `compute_partitions_avx2` currently supports only `mask`. For unsupported hashes (`mul`, `fmix64`), the AVX2 executable exits early after the runtime check. Therefore:

- `results/results.csv` contains 81 rows for `plain_novec`, 81 rows for `plain_vec`, and 27 rows for `avx2`;
- `results/benchmark_results.csv` is used for stable repeated-run analysis.

This design makes experiments reproducible and auditable: build flags, execution logs, and vectorization diagnostics are persisted and tied to each benchmark batch.

# 2 SIMD Optimization

## 2.1 Compiler evidence of vectorization

Vectorization is checked through GCC diagnostics generated at build time. In the Makefile, `plain_vec` and `avx2` are compiled with:

- `-mavx2`,
- `-fopt-info-vec-optimized -fopt-info-vec-missed`,

while, `plain_novec` is compiled with:

- `-fno-tree-vectorize`.

The following lines in `results/gcc_vec_report.txt` prove auto-vectorization for the `plain_vec` build:

```
lib/partition.cpp:19:14: optimized: loop vectorized using 16 byte vectors
lib/partition.cpp:13:20: optimized: basic block part vectorized using 32 byte vectors
```

and the manual vectorization in the AVX2 path:

```
lib/partition_avx2.cpp:44:14: optimized: loop vectorized using 16 byte vectors
lib/partition_avx2.cpp:29:27: missed: couldn't vectorize loop
```

The missed one is the scalar tail of theAVX2 kernel, which is expected since doesn't fit the SIMD register width.

## 2.2 AVX2 mask hash implementation

The AVX2 partition kernel is implemented in `lib/partition_avx2.cpp`. The logic is the following:

1. Build SIMD constants: `mask_vec = broadcast(P-1)` and `perm_idx = [0,2,4,6,...]`.
2. Process four 64-bit keys per iteration (`__m256i` register):
    - load 256 bits with `_mm256_loadu_si256`,

- compute partition ids via bitwise AND (`_mm256_and_si256`),
- pack low 32-bit lanes with `_mm256_permutevar8x32_epi32`,
- store 4 contiguous `uint32_t` outputs with `_mm_storeu_si128`.

3. Handle the tail ($< 4$ keys) with scalar code to preserve correctness for any input length.

Why this structure is effective:

- The `mask` hash is arithmetic-light and maps naturally to SIMD bitwise operations.
- Writing packed `uint32_t` partition ids reduces store bandwidth compared with 64-bit outputs.
- A scalar tail keeps control-flow simple and avoids masked-store complexity for short remainders.

Overall, this AVX2 implementation reduces partition time by combining regular memory access with low-latency SIMD integer operations.

# 3 Results Analysis

All quantitative values in this section are computed from `results/benchmark_results.csv`, because it is based on repeated runs and is more reliable than a single sweep.

## 3.1 Throughput

Figure 1 compares throughput over `P` for the three hash functions.

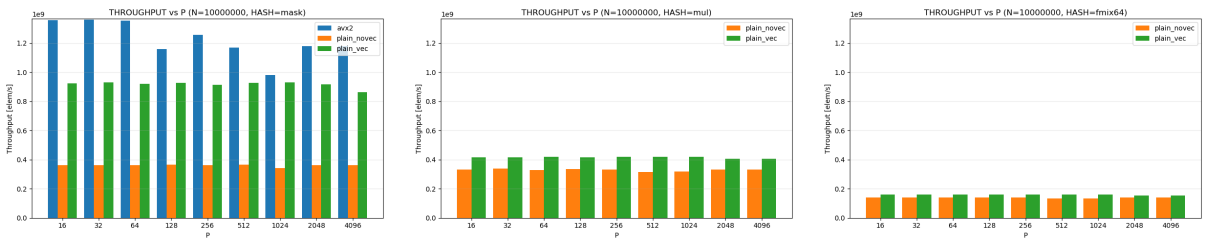


Figure 1: Throughput vs `P` for hash functions `mask`, `mul`, and `fmix64`.

The measured ordering is stable: `mask` is fastest, `mul` is second, `fmix64` is slowest. On average over `benchmark_results.csv`, throughput is:

- `plain_novec`: $3.60 \times 10^8$ elems/s (`mask`), $3.32 \times 10^8$ elems/s (`mul`), $1.40 \times 10^8$ elems/s (`fmix64`),
- `plain_vec`: $8.98 \times 10^8$ elems/s (`mask`), $4.13 \times 10^8$ elems/s (`mul`), $1.58 \times 10^8$ elems/s (`fmix64`),
- `avx2` (`mask` only): $1.28 \times 10^9$ elems/s.

This reflects the arithmetic complexity of the hash: `mask` is a simple bit operation, while `fmix64` includes multiple shifts/xors/multiplications.

## 3.2 Partition-time

Figure 2 shows partition time for each hash.

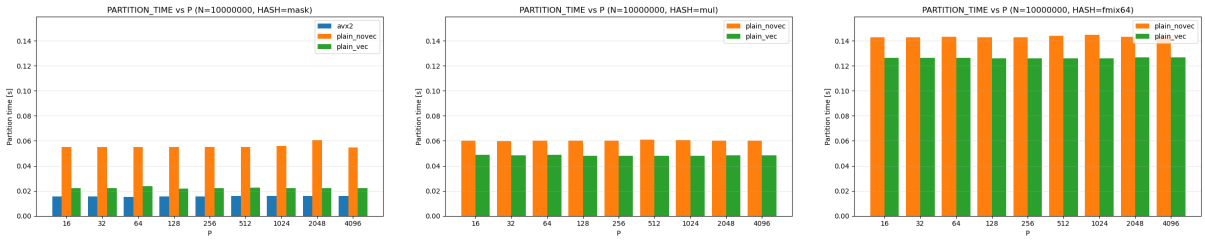


Figure 2: Partition time vs `P` for hash functions `mask`, `mul`, and `fmix64`.

Average partition time from `benchmark_results.csv` confirms the same ranking:

- `plain_novec`: 0.05571 s (`mask`), 0.06023 s (`mul`), 0.14330 s (`fmix64`),
- `plain_vec`: 0.02237 s (`mask`), 0.04842 s (`mul`), 0.12624 s (`fmix64`),
- `avx2` (`mask` only): 0.01568 s.

The impact of `P` is visible but weaker than the impact of hash complexity and SIMD level. In other words, choosing the right execution kernel and hash function yields larger gains than fine-tuning `P` alone.

## 3.3 Speedup

Figure 3 reports speedup over the scalar baseline (`plain_novec`).

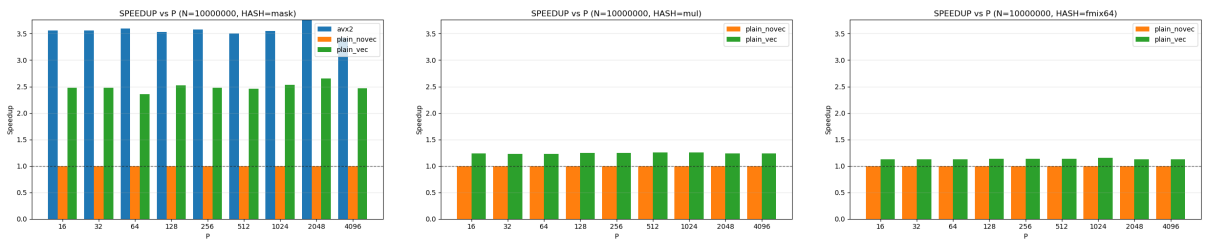


Figure 3: Speedup vs `P` (baseline: `plain_novec`) for hash functions `mask`, `mul`, and `fmix64`.

For `mul` and `fmix64`, AVX2 is currently unsupported; speedup is therefore driven by `plain_vec` only. Table 1 summarizes the `mask`-only repeated benchmark. Speedup is computed as:

$$\text{speedup}(\text{exec_type}) = \frac{\overline{T}_{\texttt{plain_novec}}}{\overline{T}_{\texttt{exec_type}}}$$

| `exec_type` | `partition_time` [s] | `throughput` [elems/s] | `speedup` |
|---|---|---|---|
| `plain_novec` | 0.055712889 | 360389759.942 | 1.0000 |
| `plain_vec` | 0.022372444 | 897678323.000 | 2.4902 |
| `avx2` | 0.015679778 | 1283223576.758 | 3.5532 |

Table 1: Average performance for `mask` over repeated runs (`benchmark_results.csv`).

# 4 Conclusions

Partition performance is driven by both algorithmic parameters (`hash`, `P`) and execution strategy (`plain_novec`, `plain_vec`, `avx2`).

SIMD is the main improvement lever: on `mask`, `plain_vec` reaches about 2.49× speedup and `avx2` about 3.55× over the scalar baseline.

Among hash functions, `mask` is consistently the fastest, `mul` is intermediate, and `fmix64` is the slowest due to higher arithmetic cost.